Secure Login

Projektdokumentation · Anmeldesystem mit PHP, MariaDB und Docker

1. Ziel des Projekts

Secure Login ist eine Webanwendung für Registrierung und Anmeldung. Der Schwerpunkt liegt auf Sicherheit: Passwörter werden nie im Klartext gespeichert, jedes Formular ist gegen Cross-Site-Request-Forgery geschützt, wiederholte Versuche werden begrenzt, und das Konto lässt sich zusätzlich mit einem Code aus dem Postfach oder einer Authenticator-App absichern. Die Anwendung ist in reinem PHP 8.3 geschrieben, bewusst einfach gehalten (13 kleine Klassen, rund 1.200 Zeilen in `src/`) und läuft komplett in Docker.

2. Funktionsumfang

Funktion	Beschreibung
Registrierung	E-Mail und Passwort ab 12 Zeichen, doppelte Adressen werden abgefangen
Login	Passwortprüfung mit Versuchsbegrenzung (20 Versuche / 15 Min. pro IP)
Bestätigung per E-Mail	Zusätzlicher sechsstelliger Code aus dem Postfach, 10 Minuten gültig, 5 Fehlversuche erlaubt. Vor dem Einschalten wird die Adresse durch einen Code bestätigt.
Zwei-Faktor-Authentifizierung	Einmalcode aus einer Authenticator-App (RFC 6238), Einrichtung per QR-Code, jeder Code gilt nur einmal
Passwort vergessen	Einmal-Link per E-Mail, 30 Minuten gültig, antwortet nie, ob ein Konto existiert
Konto-Seite	E-Mail-Bestätigung und 2FA ein- und ausschalten
Abmelden	Sitzung wird vollständig zerstört, Cookie gelöscht
Entwicklungsmodus	Ohne Mail-Zugang erscheinen Codes und Links direkt in der Seite und im Log

Die drei Anmeldevarianten lassen sich kombinieren: 1) E-Mail und Passwort, 2) zusätzlich ein Code aus dem Postfach, 3) zusätzlich ein Code aus der Authenticator-App.

3. Technischer Aufbau

Baustein	Technik	Aufgabe
Anwendung	PHP 8.3 mit Apache	Seiten ausliefern, Logik
Datenbank	MariaDB 11	Konten, Codes, Reset-Links, 2FA-Geheimnisse, Zähler
Oberfläche	HTML in PHP-Templates, CSS, etwas JavaScript	Formulare und QR-Code
E-Mail	SMTP über PHPMailer (z. B. Resend)	Codes und Reset-Link versenden
Betrieb	Docker Compose, 2 Container (app, db)	Start mit einem Befehl

Als einzige Bibliothek kommt PHPMailer über Composer. Alles andere (Einmalcodes nach RFC 6238, Verschlüsselung mit libsodium, Versuchsbegrenzung) ist mit PHP-Bordmitteln umgesetzt.

4. Die wichtigsten Dateien

Nur der Ordner `public/` ist über den Browser erreichbar. Dort liegen die Seiten, die man aufruft. Die Logik in `src/`, die Vorlagen und die Konfiguration sind von außen nicht erreichbar.

Anwendungslogik (src/)

Datei	Zeilen	Aufgabe
Application.php	40	Baut alle Bausteine einmal zusammen, die Seiten holen sich hier, was sie brauchen
Web.php	162	Eingaben, Sitzung, CSRF-Schutz, Weiterleitung, Meldungen, Rendern der Templates
Auth.php	160	Angemeldeter Zustand und mehrstufiger Login (Passwort, dann E-Mail-Code, dann App-Code)
EmailCodes.php	119	Codes per E-Mail: erzeugen, versenden, prüfen, Fehlversuche zählen
TwoFactor.php	116	Authenticator-App einrichten, Code prüfen, 2FA ausschalten
Totp.php	92	Berechnung der Einmalcodes nach RFC 6238 (HMAC-SHA1, 30 Sekunden, 6 Stellen)
PasswordReset.php	90	Reset-Link erzeugen und einlösen
Mailer.php	93	E-Mail-Versand über SMTP, im Entwicklungsmodus Anzeige statt Versand
Security.php	89	Passwort-Hashing, Verschlüsselung, Eingabeprüfung für E-Mail und Passwort
Database.php	90	PDO mit Prepared Statements, Transaktionen
Users.php	58	Zugriff auf die Tabelle users
Config.php · RateLimit.php	48 · 35	Konfiguration aus .env · Versuchsbegrenzung

Seiten, Vorlagen und Weiteres

Ort	Inhalt
public/*.php	Eine kurze Datei je Seite (10 bis 100 Zeilen): login, register, account, verify-email, verify-2fa, forgot-password, reset-password, logout. Sie lesen die Eingabe, rufen die Logik auf und geben die Seite aus.
public/assets/	style.css, account.js (zeigt den QR-Code), QR-Code-Bibliothek
templates/	HTML der Seiten als PHP-Dateien, layout.php als gemeinsamer Rahmen, emails/ für die Mail-Vorlagen
database/schema.sql	Fünf Tabellen, wird beim ersten Start automatisch eingelesen
Dockerfile · docker-compose.yml	Image mit Apache und PHP, Dienste app und db
tests/run.php	16 Tests ohne Datenbank (u. a. offizieller RFC-6238-Testvektor)
.env.example	Vorlage für die Konfiguration, die echte .env gehört nie ins Git

5. Ablauf einer Anmeldung

- Der Benutzer sendet E-Mail und Passwort an `/login.php` (per HTTPS im Betrieb).
- Der Server prüft das Passwort gegen den bcrypt-Hash in der Datenbank.
- Hat das Konto die E-Mail-Bestätigung eingeschaltet, wird ein Code per Mail verschickt und auf `/verify-email.php` abgefragt.

- Hat das Konto die 2FA eingeschaltet, folgt `/verify-2fa.php` mit dem Code aus der App.
- Erst wenn alle Stufen bestanden sind, bekommt der Benutzer eine neue Sitzungs-ID und landet auf `/account.php`. Bis dahin steht das Konto nur als „wartend“ in der Sitzung (15 Minuten lang) und hat keinen Zugriff.

6. Sicherheitskonzept

Angriff	Gegenmaßnahme
SQL-Injection	Ausschließlich Prepared Statements, keine emulierten Statements
Cross-Site-Scripting	Jede Ausgabe in den Templates läuft durch htmlspecialchars, zusätzlich Content-Security-Policy ohne Inline-Skripte
CSRF	Verstecktes Token (32 Zufallsbytes) in jedem Formular, zeitkonstant verglichen
Brute-Force	Zähler in der Datenbank: 20 Logins / 15 Min., 10 App-Codes / 15 Min., 5 Reset-Anfragen / Std.; E-Mail-Codes werden nach 5 Fehlversuchen ungültig
Passwort im Klartext	bcrypt mit Salt (password_hash), Mindestlänge 12 Zeichen
Session-Diebstahl	HttpOnly, SameSite=Lax, Secure bei HTTPS, neue Sitzungs-ID nach dem Login; die Sitzung ist an Passwort und 2FA-Geheimnis gebunden und wird bei einer Änderung sofort ungültig
Wiederverwendeter App-Code	Jeder 30-Sekunden-Schritt gilt nur einmal (last_used_step, atomar per UPDATE)
Datenbank-Diebstahl	Mail-Codes nur als HMAC, Reset-Token nur als SHA-256-Hash, 2FA-Geheimnis verschlüsselt (libsodium secretbox mit APP_KEY)
Doppelt eingelöster Reset-Link	Transaktion mit SELECT … FOR UPDATE
Konten ausspähen	„Passwort vergessen“ antwortet immer gleich
Clickjacking u. a.	X-Frame-Options, nosniff, Referrer-Policy; PHP zeigt keine Fehler im Browser

7. Datenbank

Tabelle	Inhalt
users	E-Mail (eindeutig), Passwort-Hash, Schalter für die E-Mail-Bestätigung, letzter Login
email_verifications	Hash des Codes, Zweck (Login oder Adresse), Ablaufzeit, Fehlversuche
password_resets	Hash des Tokens, Ablaufzeit
user_totp	Verschlüsseltes 2FA-Geheimnis, zuletzt genutzter Zeitschritt
auth_rate_limits	Zähler und Ablaufzeit für die Versuchsbegrenzung

8. Inbetriebnahme und Tests

```
cp .env.example .env
APP_KEY in .env eintragen (php -r 'echo base64_encode(random_bytes(32));')
docker compose up -d --build
```

Die App läuft dann auf **http://127.0.0.1:8081**. Ohne Mail-Zugang (SMTP_PASSWORD leer) gilt der Entwicklungsmodus. Für echte Mails werden SMTP_USER, SMTP_PASSWORD und SMTP_FROM in der .env gesetzt (z. B. Resend). Die Tests starten mit `composer test`.

Wichtig: Der APP_KEY darf nach dem ersten Start nicht mehr geändert werden, sonst lassen sich gespeicherte 2FA-Geheimnisse nicht mehr entschlüsseln.

9. Bekannte Grenzen und mögliche Erweiterungen

- Die Versuchsbegrenzung zählt pro IP-Adresse. Hinter einem Proxy müsste die echte Client-IP ausgewertet werden.
- Es gibt keine Wiederherstellungscodes für die 2FA. Geht das Handy verloren, muss die 2FA in der Datenbank entfernt werden.
- Es gibt keine automatischen Tests für den kompletten Ablauf mit Datenbank, nur Tests der Bausteine. Der Ablauf wurde im laufenden Stack geprüft.
- Mögliche Erweiterungen: Wiederherstellungscodes, Passwort ändern auf der Konto-Seite, Konto löschen, mehr automatische Tests.